

Codebase Architect Explainer

1) Big Picture

This project is a Node.js Express backend API focused on authentication and user management. It provides health endpoints, request validation, user signup persistence in PostgreSQL, and JWT cookie issuance.

2) Core Architecture

The codebase follows a layered monolith architecture:

- Entry/bootstrap: `src/index.js` -> `src/server.js` -> `src/app.js`
- HTTP transport: `src/routes`
- Controller layer: `src/controllers`
- Service/business layer: `src/services`
- Persistence/schema: `src/models` + `src/config/database.js` + drizzle migrations
- Shared utilities and validation: `src/utils` + `src/validations` + `src/config/logger.js`

High-level flow:

Client Request

- > Express middleware stack in `app.js`
- > `/api/auth` routes
- > controller validation and orchestration
- > service DB operations via Drizzle
- > response + cookie + logs

3) Key Components

- `src/app.js`: middleware, logging, health routes, API route mounting
- `src/routes/auth.routes.js`: route definitions
- `src/controllers/auth.controller.js`: signup request handling
- `src/services/auth.service.js`: user lookup/create and password hashing
- `src/models/user.model.js`: users table schema
- `src/validations/auth.validation.js`: Zod input schemas
- `src/utils/jwt.js` and `src/utils/cookies.js`: token and cookie helpers
- `src/config/logger.js`: Winston logging setup
- `src/config/database.js`: Neon + Drizzle database connection

4) Data Flow and Communication

1. Request enters Express middleware.
2. Route dispatches to auth controller.
3. Controller validates payload with Zod.
4. Controller calls service.
5. Service queries/inserts using Drizzle against Postgres (Neon).
6. Controller signs JWT, sets HTTP-only cookie, and returns JSON.
7. Morgan and Winston capture logs.

5) Tech Stack and Dependencies

- express: HTTP server and routing
- drizzle-orm + drizzle-kit: data access and migrations
- @neondatabase/serverless: PostgreSQL connectivity

- zod: request validation
- bcrypt: password hashing
- jsonwebtoken: token generation and verification
- helmet, cors, cookie-parser: API security and cookie parsing
- morgan + winston: request and structured logging

6) Typical Execution Flow (Signup)

User sends POST /api/auth/sign-up with name/email/password.

- > signup schema validates the body.
- > createUser checks for existing email.
- > password is hashed with bcrypt.
- > user row is inserted into users table.
- > JWT token is generated.
- > token is set in an HTTP-only cookie.
- > API responds with 201 and user payload.

7) Strengths and Tradeoffs

Strengths:

- Clear layered boundaries (route/controller/service/model/validation).
- Security baseline present (hashing, JWT, HTTP-only cookies, helmet).
- Migration-backed relational schema with structured logging.

Tradeoffs / watchouts:

- sign-in and sign-out are still placeholders.
- Error handling is not centralized via a global error middleware.
- A few naming and consistency issues can cause runtime bugs.
- Email uniqueness is enforced in code but not yet at DB constraint level.

8) Final Summary

This is a layered Express monolith that implements the foundation of an auth API.

Requests flow from routes to controllers to services to Drizzle/Postgres, with Zod validation,

JWT cookies, and Winston logging across the stack. It is a solid starter architecture with clear

separation of concerns, but auth completeness and production hardening still need follow-up work.